
Sockets and Patterns

In [Chapter 1](#) we took ØMQ for a drive, with some basic examples of the main ØMQ patterns: request-reply, publish-subscribe, and pipeline. In this chapter, we're going to get our hands dirty and start to learn how to use these tools in real programs.

We'll cover:

- How to create and work with ØMQ sockets
- How to send and receive messages on sockets
- How to build your apps around ØMQ's asynchronous I/O model
- How to handle multiple sockets in one thread
- How to handle fatal and nonfatal errors properly
- How to handle interrupt signals like Ctrl-C
- How to shut down a ØMQ application cleanly
- How to check a ØMQ application for memory leaks
- How to send and receive multipart messages
- How to forward messages across networks
- How to build a simple message queuing broker
- How to write multithreaded applications with ØMQ
- How to use ØMQ to signal between threads
- How to use ØMQ to coordinate a network of nodes
- How to create and use message envelopes for publish-subscribe
- How to use the high-water mark (HWM) to protect against memory overflows

The Socket API

To be perfectly honest, ØMQ does a kind of switch-and-bait on you, for which we don't apologize. It's for your own good, and it hurts us more than it hurts you. It presents a familiar socket-based API, which requires great effort for us to hide a bunch of message-processing engines. However, the result will slowly fix your world view about how to design and write distributed software.

Sockets are the de facto standard API for network programming, as well as being useful for stopping your eyes from falling onto your cheeks. One thing that makes ØMQ especially tasty to developers is that it uses sockets and messages instead of some other arbitrary set of concepts. Kudos to Martin Sustrik for pulling this off. It turns “Message Oriented Middleware,” a phrase guaranteed to send the whole room off to Catatonia, into “Extra Spicy Sockets!,” which leaves us with a strange craving for pizza and a desire to know more.

Like a favorite dish, ØMQ sockets are easy to digest. **These sockets have a life in four parts**, just like BSD sockets:

1. • We can create and destroy them, which go together to form a karmic circle of socket life (see `zmq_socket()`, `zmq_close()`).
2. • We can configure them by setting options on them and checking them if necessary (see `zmq_setsockopt()`, `zmq_getsockopt()`).
3. • We can plug them into the network topology by creating ØMQ connections to and from them (see `zmq_bind()`, `zmq_connect()`).
4. • We can use them to carry data by writing and receiving messages on them (see `zmq_msg_send()`, `zmq_msg_recv()`).

Note that **sockets are always void pointers**, and **messages** (which we'll come to very soon) **are structures**. So in C you pass sockets as such, but you pass addresses of messages in all functions that work with messages, like `zmq_msg_send()` and `zmq_msg_recv()`. As a mnemonic, realize that “**in ØMQ, all your sockets are belong to us,**” but **messages are things you actually own in your code.**

Creating, destroying, and configuring sockets works as you'd expect for any object. But remember that ØMQ is an asynchronous, elastic fabric. This has some impact on how we plug sockets into the network topology and how we use the sockets after that.

Plugging Sockets into the Topology

To create a connection between two nodes, you use `zmq_bind()` in one node and `zmq_connect()` in the other. As a general rule of thumb, the **node that does `zmq_bind()`** is a “server,” sitting on a well-known network address, and the **node that does `zmq_connect()`** is a “client,” with unknown or arbitrary network addresses. Thus, we say that we

“bind a socket to an endpoint” and “connect a socket to an endpoint,” the endpoint being that well-known network address.

ØMQ connections are somewhat different from old-fashioned TCP connections. The main notable differences are:

Diff between ZermMQ and TCP connection

- They go across an arbitrary transport (inproc, ipc, tcp, pgm, or epgm). See `zmq_inproc()`, `zmq_ipc()`, `zmq_tcp()`, `zmq_pgm()`, and `zmq_epgm()`.
- One socket may have many outgoing and many incoming connections.
- There is no `zmq_accept()` method. When a socket is bound to an endpoint, it automatically starts accepting connections.
- The network connection itself happens in the background, and ØMQ will automatically reconnect if the network connection is broken (e.g., if the peer disappears and then comes back).
- Your application code cannot work with these connections directly; they are encapsulated under the socket.

Many architectures follow some kind of client/server model, where the server is the component that is most static, and the clients are the components that are most dynamic—i.e., they come and go the most. There are sometimes issues of addressing: servers will be visible to clients, but not necessarily vice versa. So mostly it's obvious which node should be doing `zmq_bind()` (the server) and which should be doing `zmq_connect()` (the client). It also depends on the kind of sockets you're using, with some exceptions for unusual network architectures. We'll look at socket types later.

服务器也能连

Now, imagine we start the client *before* we start the server. In traditional networking, we'd get a big red Fail flag. But ØMQ lets us start and stop pieces arbitrarily. As soon as the client node does `zmq_connect()`, the connection exists and that node can start to write messages to the socket. At some stage (hopefully before messages queue up so much that they start to get discarded, or the client blocks), the server comes alive, does a `zmq_bind()`, and ØMQ starts to deliver messages.

A server node can bind to many endpoints (that is, a combination of protocol and address), and it can do this using a single socket. This means it will accept connections across different transports:

```
zmq_bind (socket, "tcp://*:5555");
zmq_bind (socket, "tcp://*:9999");
zmq_bind (socket, "inproc://somename");
```

With most transports, you cannot bind to the same endpoint twice (unlike, for example, in UDP). The `ipc` transport does, however, let one process bind to an endpoint already used by another process. It's meant to allow a process to recover after a crash.

Although ØMQ tries to be neutral about which side binds and which side connects, there are differences. We'll see these in more detail later. The upshot is that you should usually think in terms of “servers” as static parts of your topology that bind to more or less fixed endpoints, and “clients” as dynamic parts that come and go and connect to these endpoints. Then, design your application around this model. The chances that it will “just work” are much better like that.

Sockets have types. The socket type defines the semantics of the socket: its policies for routing messages inwards and outwards, queuing, etc. You can connect certain types of socket together, such as a publisher socket and a subscriber socket. Sockets work together in “messaging patterns.” We'll look at this in more detail later.

It's the ability to connect sockets in these different ways that gives ØMQ its basic power as a message queuing system. There are layers on top of this, such as proxies, which we'll get to later. But essentially, with ØMQ you define your network architecture by plugging pieces together like a child's construction toy.

Using Sockets to Carry Data

To send and receive messages, you use the `zmq_msg_send()` and `zmq_msg_recv()` methods. The names are conventional, but ØMQ's I/O model is different enough from the TCP model (Figure 2-1) that you will need time to get your head around it.

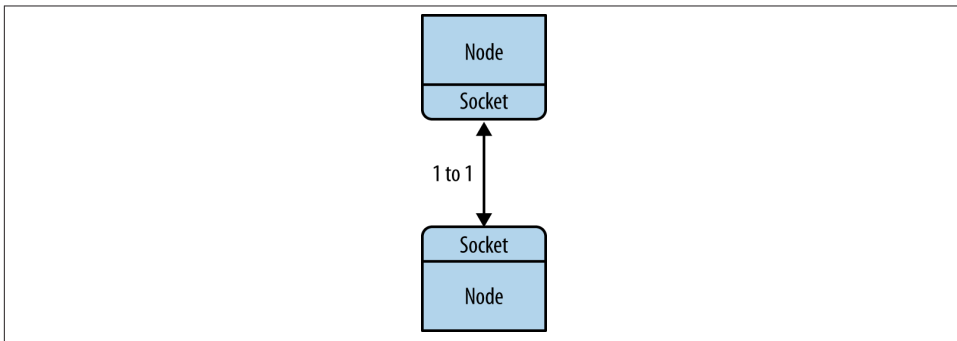


Figure 2-1. TCP sockets are 1-to-1

Let's look at the **main differences between TCP sockets and ØMQ sockets** when it comes to working with data:

- **ØMQ sockets carry messages, like UDP**, rather than a stream of bytes as TCP does. A **ØMQ message is length-specified binary data**. We'll come to messages shortly; their design is optimized for performance and so a little tricky.

- ØMQ sockets do their I/O in a background thread. This means that messages arrive in local input queues and are sent from local output queues, no matter what your application is busy doing.
- ØMQ sockets have 1-to-N routing behavior built in, according to the socket type.

不會馬上送

The `zmq_msg_send()` method does not actually send the message to the socket connection(s). It queues the message so that the I/O thread can send it asynchronously. It does not block except in some exception cases. So, the message is not necessarily sent when `zmq_msg_send()` returns to your application.

Unicast Transports → 3種: TCP, ipc, inproc

ØMQ provides a set of unicast transports (`inproc`, `ipc`, and `tcp`) and multicast transports (`epgm`, `pgm`). Multicast is an advanced technique that we'll come to later. Don't even start using it unless you know that your fan-out ratios will make 1-to-N unicast impossible.

For most common cases, use `tcp`, which is a *disconnected* TCP transport. It is elastic, portable, and fast enough for most cases. We call this “disconnected” because ØMQ's `tcp` transport doesn't require the endpoint to exist before you connect to it. Clients and servers can connect and bind at any time, can go and come back, and it remains transparent to applications.

The inter-process `ipc` transport is also *disconnected*. It has one limitation: it does not yet work on Windows. By convention we use endpoint names with an “.ipc” extension to avoid potential conflict with other filenames. On Unix systems, if you use `ipc` endpoints you need to create these with appropriate permissions; otherwise, they may not be shareable between processes running under different user IDs. You must also make sure all processes can access the files, e.g., by running in the same working directory.

The inter-thread transport, `inproc`, is a *connected* signaling transport. It is much faster than `tcp` or `ipc`. This transport has a specific limitation compared to `tcp` and `ipc`, however *the server must issue a bind request before any client issues a connect*. This is something future versions of ØMQ may fix, but at present this defines how you use `inproc` sockets. We create and bind one socket and start the child threads, which create and connect the other sockets.

ØMQ Is Not a Neutral Carrier

A common question that newcomers to ØMQ ask (it's one I've asked myself) is, “How do I write an XYZ server in ØMQ?” For example, “How do I write an HTTP server in ØMQ?” The implication is that if we use normal sockets to carry HTTP requests and responses, we should be able to use ØMQ sockets to do the same, only much faster and better.

ZeroMQ Framing

The answer used to be, “This is not how it works.” ØMQ is not a neutral carrier; it imposes a framing on the transport protocols it uses. This framing is not compatible with existing protocols, which tend to use their own framing. For example, compare an HTTP request (Figure 2-2) and a ØMQ request (Figure 2-3), both over TCP/IP. The HTTP request uses CRLF (carriage return line feed) as its simplest framing delimiter, whereas ØMQ uses a length-specified frame.

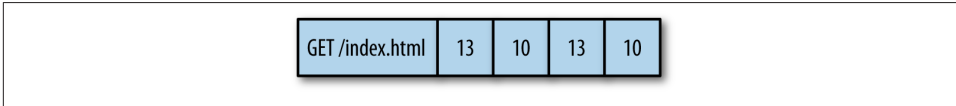


Figure 2-2. HTTP on the wire

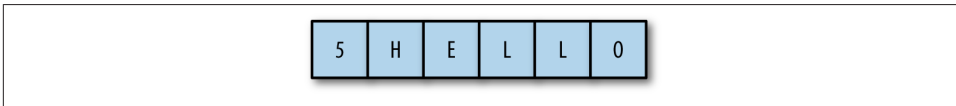


Figure 2-3. ØMQ on the wire

So, you could write an HTTP-like protocol using ØMQ, using for example the request-reply socket pattern. But it would not be HTTP.

Since ØMQ v3.3, however, ØMQ has a socket option called `ZMQ_ROUTER_RAW` that lets you read and write data without the ØMQ framing. You could use this to read and write proper HTTP requests and responses. Hardeep Singh contributed this change so that he could connect to Telnet servers from his ØMQ application. This is still, at the time of writing, somewhat experimental, but it shows how ØMQ keeps evolving to solve new problems. Maybe the next patch will be yours.

I/O Threads

We said that ØMQ does I/O in a background thread. One I/O thread (for all sockets) is sufficient for all but the most extreme applications. When you create a new context, it starts with one I/O thread. The general rule of thumb is to allow one I/O thread per gigabyte of data in or out per second. To raise the number of I/O threads, use the `zmq_ctx_set()` call before creating any sockets:

```
int io-threads = 4;
void *context = zmq_ctx_new ();
zmq_ctx_set (context, ZMQ_IO_THREADS, io_threads);
assert (zmq_ctx_get (context, ZMQ_IO_THREADS) == io_threads);
```

- 一个 thread 能处理
一个 socket 的多个
connection

We've seen that one socket can handle dozens, even thousands of connections at once. This has a fundamental impact on how you write applications. A traditional networked application has one process or one thread per remote connection, and that process or

传统: 一个 connection 配一个 process
即一个 socket

thread handles one socket. ØMQ lets you collapse this entire structure into a single process and then break it up as necessary for scaling.

If you are using ØMQ for inter-thread communications only (i.e., a multithreaded application that does no external socket I/O) you can set the I/O threads to zero. It's not a significant optimization though, more of a curiosity.

Messaging Patterns

Underneath the brown paper wrapping of ØMQ's socket API lies the world of messaging patterns. If you have a background in enterprise messaging, or know UDP well, these will be vaguely familiar. But to most ØMQ newcomers, they are a surprise because we're so used to the TCP paradigm where a socket maps one-to-one to another node.

Let's recap briefly what ØMQ does for you. It delivers blobs of data (messages) to nodes, quickly and efficiently. You can map nodes to threads, processes, or nodes. ØMQ gives your applications a single socket API to work with, no matter what the actual transport is (e.g., in-process, inter-process, TCP, or multicast). It automatically reconnects to peers as they come and go. It queues messages at both sender and receiver, as needed. It manages these queues carefully to ensure processes don't run out of memory, overflowing to disk when appropriate. It handles socket errors. It does all I/O in background threads. It uses lock-free techniques for talking between nodes, so there are never locks, waits, semaphores, or deadlocks.

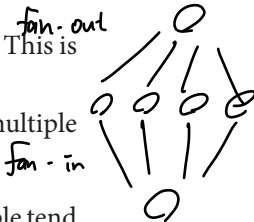
pattern

But cutting through that, it routes and queues messages according to precise recipes called *patterns*. It is these patterns that provide ØMQ's intelligence. They encapsulate our hard-earned experience of the best ways to distribute data and work. ØMQ's patterns are hard-coded, but future versions may allow user-definable patterns.

ØMQ patterns are implemented by pairs of sockets with matching types. In other words, to understand ØMQ patterns you need to understand socket types and how they work together. Mostly, this just takes some studying; there is little that is obvious at this level.

The built-in core ØMQ patterns are:

1. *Request-reply*, which connects a set of clients to a set of services. This is a remote procedure call and task distribution pattern.
2. *Publish-subscribe*, which connects a set of publishers to a set of subscribers. This is a data distribution pattern.
3. *Pipeline*, which connects nodes in a fan-out/fan-in pattern that can have multiple steps and loops. This is a parallel task distribution and collection pattern.



We looked at each of these in the first chapter. There's one more pattern that people tend to try to use when they still think of ØMQ in terms of traditional TCP sockets: *exclusive*

pair, which connects two sockets exclusively. This is a pattern you should use only to connect two threads in a process. We'll see an example at the end of this chapter.

The `zmq_socket()` man page is fairly clear about the patterns; it's worth reading it several times until it starts to make sense. These are the socket combinations that are valid for a connect-bind pair (either side can bind):

- 
- PUB and SUB
 - REQ and REP
 - REQ and ROUTER
 - DEALER and REP
 - DEALER and ROUTER
 - DEALER and DEALER
 - ROUTER and ROUTER
 - PUSH and PULL
 - PAIR and PAIR

You'll also see references to XPUB and XSUB sockets, which we'll come to later (they're like raw versions of PUB and SUB). Any other combination will produce undocumented and unreliable results, and future versions of ØMQ will probably return errors if you try them. You can and will, of course, bridge other socket types via code (i.e., read from one socket type and write to another).

High-Level Messaging Patterns

These four core patterns are cooked into ØMQ. They are part of the ØMQ API, implemented in the core C++ library, and are guaranteed to be available in all fine retail stores.

On top of those, we add *high-level patterns*. We build these high-level patterns on top of ØMQ and implement them in whatever language we're using for our application. They are not part of the core library, do not come with the ØMQ package, and exist in their own space as part of the ØMQ community. For example, the Majordomo pattern, which we explore in Chapter 4 sits in the GitHub Majordomo project in the ØMQ organization.

One of the things we aim to provide you with in this book is a set of such high-level patterns, both small (how to handle messages sanely) and large (how to make a reliable publish-subscribe architecture).

Working with Messages 任意size的資料塊

On the wire, ØMQ messages are blobs of any size, from zero upwards, that fit in memory. You do your own serialization using protobufs, msgpack, JSON, or whatever else your applications need to speak. It's wise to choose a data representation that is portable and fast, but you can make your own decisions about trade-offs.

In memory, ØMQ messages are `zmq_msg_t` structures (or classes, depending on your language). Here are the basic ground rules for using ØMQ messages in C:

做各個 message work 所用的API

for create msg

for receive

for send

for release msg

for seize content

- You create and pass around `zmq_msg_t` objects, not blocks of data.
- To read a message, you use `zmq_msg_init()` to create an empty message, and then you pass that to `zmq_msg_recv()`.
- To write a message from new data, you use `zmq_msg_init_size()` to create a message and at the same time allocate a block of data of some size. You then fill that data using `memcpy()`, and pass the message to `zmq_msg_send()`.
- To release (not destroy) a message, you call `zmq_msg_close()`. This drops a reference, and eventually ØMQ will destroy the message.
- To access the message content, you use `zmq_msg_data()`. To know how much data the message contains, use `zmq_msg_size()`.
- Do not use `zmq_msg_move()`, `zmq_msg_copy()`, or `zmq_msg_init_data()` unless you've read the manual pages and know precisely why you need these.

Here is a typical chunk of code working with messages that should be familiar if you have been paying attention. This is from the `zhelpers.h` file we use in all the examples:

```
// Receive ØMQ string from socket and convert into C string
static char *
s_recv (void *socket) {
    zmq_msg_t message;
    zmq_msg_init (&message);
    int size = zmq_msg_recv (&message, socket, 0);
    if (size == -1)
        return NULL;
    char *string = malloc (size + 1);
    memcpy (string, zmq_msg_data (&message), size);
    zmq_msg_close (&message);
    string [size] = 0;
    return (string);
}

// Convert C string to ØMQ string and send to socket
static int
s_send (void *socket, char *string) {
    zmq_msg_t message;
    zmq_msg_init_size (&message, strlen (string));
```

```

memcpy (zmq_msg_data (&message), string, strlen (string));
int size = zmq_msg_send (&message, socket, 0);
zmq_msg_close (&message);
return (size);
}

```

You can easily extend this code to send and receive blobs of arbitrary length.



After you pass a message to `zmq_msg_send()`, ØMQ will clear the message (i.e., set the size to zero). You cannot send the same message twice, and you cannot access the message data after sending it.

If you want to send the same message more than once, create a second message, initialize it using `zmq_msg_init()`, and then use `zmq_msg_copy()` to create a copy of the first message. This does not copy the data, but rather the reference. You can then send the message twice (or more, if you create more copies), and the message will only be finally destroyed when the last copy is sent or closed.

ØMQ also supports *multipart* messages, which let you send or receive a list of frames as a single on-the-wire message. This is widely used in real applications, as we'll see later in this chapter and in **Chapter 3**.

Def of Frame

Frames (also called “message parts” in the ØMQ reference manual pages) are the basic wire format for ØMQ messages. A frame is a length-specified block of data. The length can be from zero upwards. If you've done any TCP programming you'll appreciate why frames are a useful answer to the question, “How much data am I supposed to read off this network socket now?”



There is a **wire-level protocol called ZMTP** that defines how ØMQ reads and writes frames on a TCP connection. If you're interested in how this works, the spec is quite short.

把 message 拆成多个 part (frame)

Originally, a ØMQ message was one frame, like UDP. We later **extended this with multipart messages**, which are quite **simply series of frames** with a “more” bit set to one, followed by one with that bit set to zero. The ØMQ API then lets you write messages with a “more” flag, and when you read messages, it lets you check if there's “more.”

In the low-level ØMQ API and the reference manual, therefore, there's some fuzziness about messages versus parts. So here's a useful lexicon:

1. • A message can be one or more parts.
2. • These parts are also called *frames*.

low-level 如何分辨 messages 和 part?

3. Each part is a `zmq_msg_t` object.
4. You send and receive each part separately, in the low-level API.
5. Higher-level APIs provide wrappers to send entire multipart messages.

Some other things that are worth knowing about messages:

- You may send zero-length messages, e.g., for sending a signal from one thread to another.
- ØMQ guarantees to deliver all the parts (one or more) of a message, or none of them.
- ØMQ does not send the message (single or multipart) right away, but at some indeterminate later time. A multipart message must therefore fit in memory.
- A single-part message must also fit in memory. If you want to send files of arbitrary sizes, you should break them into pieces and send each piece as separate single-part messages.
- You must call `zmq_msg_close()` when finished with a message, in languages that don't automatically destroy objects when a scope closes.

And to be necessarily repetitive, do not use `zmq_msg_init_data()` yet. This is a zero-copy method and is guaranteed to create trouble for you. There are far more important things to learn about ØMQ before you start to worry about shaving off microseconds.

Handling Multiple Sockets

The main loop of most examples so far has been:

1. Wait for message on socket.
2. Process message.
3. Repeat.

What if we want to read from multiple endpoints at the same time? The simplest way is to connect one socket to all the endpoints and get ØMQ to do the fan-in for us. This is legal if the remote endpoints are in the same pattern, but it would be wrong to connect a PULL socket to a PUB endpoint, for example.

To actually read from multiple sockets all at once, use `zmq_poll()`. An even better way might be to wrap `zmq_poll()` in a framework that turns it into a nice event-driven *reactor*, but that involves significantly more work than we want to cover here.

Let's start with a dirty hack, partly for the fun of not doing it right, but mainly because it lets me show you how to do non-blocking socket reads. Example 2-1 is a simple

example of reading from two sockets using non-blocking reads. This rather confused program acts both as a subscriber to weather updates, and a worker for parallel tasks.

Example 2-1. Multiple socket reader (msreader.c)

```
//
// Reading from multiple sockets
// This version uses a simple recv loop
//
#include "zhelpers.h"

int main (void)
{
    // Prepare our context and sockets
    void *context = zmq_ctx_new ();

    // Connect to task ventilator
    void *receiver = zmq_socket (context, ZMQ_PULL);
    zmq_connect (receiver, "tcp://localhost:5557");

    // Connect to weather server
    void *subscriber = zmq_socket (context, ZMQ_SUB);
    zmq_connect (subscriber, "tcp://localhost:5556");
    zmq_setsockopt (subscriber, ZMQ_SUBSCRIBE, "10001 ", 6);

    // Process messages from both sockets
    // We prioritize traffic from the task ventilator
    while (1) {
        // Process any waiting tasks
        int rc;
        for (rc = 0; !rc; ) {
            zmq_msg_t task;
            zmq_msg_init (&task);
            if ((rc = zmq_msg_recv (&task, receiver, ZMQ_DONTWAIT)) != -1) {
                // Process task
            }
            zmq_msg_close (&task);
        }
        // Process any waiting weather updates
        for (rc = 0; !rc; ) {
            zmq_msg_t update;
            zmq_msg_init (&update);
            if ((rc = zmq_msg_recv (&update, subscriber, ZMQ_DONTWAIT)) != -1) {
                // Process weather update
            }
            zmq_msg_close (&update);
        }
        // No activity, so sleep for 1 msec
        s_sleep (1);
    }
    // We never get here, but clean up anyhow
    zmq_close (receiver);
}
```

```

    zmq_close (subscriber);
    zmq_ctx_destroy (context);
    return 0;
}

```

The cost of this approach is some additional latency on the first message (the sleep at the end of the loop, when there are no waiting messages to process). This would be a problem in applications where submillisecond latency was vital. Also, you need to check the documentation for `nanosleep()` (or whatever function you use) to make sure it does not busy-loop.

You can treat the sockets fairly by reading first from one, then the second, rather than prioritizing them as we did in this example.

Example 2-2 shows the same little senseless application done right, using `zmq_poll()`.

Example 2-2. Multiple socket poller (mspoller.c)

```

//
// Reading from multiple sockets
// This version uses zmq_poll()
//
#include "zhelpers.h"

int main (void)
{
    void *context = zmq_ctx_new ();

    // Connect to task ventilator
    void *receiver = zmq_socket (context, ZMQ_PULL);
    zmq_connect (receiver, "tcp://localhost:5557");

    // Connect to weather server
    void *subscriber = zmq_socket (context, ZMQ_SUB);
    zmq_connect (subscriber, "tcp://localhost:5556");
    zmq_setsockopt (subscriber, ZMQ_SUBSCRIBE, "10001 ", 6);

    // Initialize poll set
    zmq_pollitem_t items [] = {
        { receiver, 0, ZMQ_POLLIN, 0 },
        { subscriber, 0, ZMQ_POLLIN, 0 }
    };

    // Process messages from both sockets
    while (1) {
        zmq_msg_t message;
        zmq_poll (items, 2, -1);
        if (items [0].revents & ZMQ_POLLIN) {
            zmq_msg_init (&message);
            zmq_msg_recv (&message, receiver, 0);
            // Process task
            zmq_msg_close (&message);
        }
    }
}

```

```

        if (items [1].revents & ZMQ_POLLIN) {
            zmq_msg_init (&message);
            zmq_msg_recv (&message, subscriber, 0);
            // Process weather update
            zmq_msg_close (&message);
        }
    }
    // We never get here
    zmq_close (receiver);
    zmq_close (subscriber);
    zmq_ctx_destroy (context);
    return 0;
}

```

The items structure has these four members:

```

typedef struct {
    void *socket;      // OMQ socket to poll on
    int fd;            // OR, native file handle to poll on
    short events;      // Events to poll on
    short revents;     // Events returned after poll
} zmq_pollitem_t;

```

Multipart Messages

ØMQ lets us compose a message out of several frames, giving us a “multipart message.” Realistic applications use multipart messages heavily, both for wrapping messages with address information and for simple serialization. We’ll look at reply envelopes later. What we’ll learn now is simply how to safely (but blindly) read and write multipart messages in any application (like a proxy) that needs to forward messages without inspecting them.

When you work with multipart messages, each part is a `zmq_msg` item. For example, if you are sending a message with five parts, you must construct, send, and destroy five `zmq_msg` items. You can do this in advance (and store the `zmq_msg` items in an array or structure), or as you send them, one by one.

Here is how we send the frames in a multipart message (we receive each frame into a message object):

```

zmq_msg_send (socket, &message, ZMQ_SNDMORE);
...
zmq_msg_send (socket, &message, ZMQ_SNDMORE);
...
zmq_msg_send (socket, &message, 0);

```

Here is how we receive and process all the parts in a message, be it single part or multipart:

```

while (1) {
    zmq_msg_t message;

```

```

    zmq_msg_init (&message);
    zmq_msg_rcv (socket, &message, 0);
    // Process the message frame
    zmq_msg_close (&message);
    int more;
    size_t more_size = sizeof (more);
    zmq_getsockopt (socket, ZMQ_RCVMORE, &more, &more_size);
    if (!more)
        break;    // Last message frame
}

```

Some things to know about multipart messages:

why?

- When you send a multipart message, the first part and all following parts are only actually sent on the wire when you send the final part.
- If you are using `zmq_poll()`, when you receive the first part of a message, all the rest has also arrived.
- You will receive all parts of a message, or none at all.
- Each part of a message is a separate `zmq_msg` item.
- You will receive all parts of a message whether or not you set the RCVMORE option.
- On sending, ØMQ queues message frames in memory until the last is received, and then sends them all.
- There is no way to cancel a partially sent message, except by closing the socket.

不管有無set RCVMORE

Intermediaries and Proxies

ØMQ aims for decentralized intelligence, but that doesn't mean your network is empty space in the middle. It's filled with message-aware infrastructure, and quite often, you build that infrastructure with ØMQ. The ØMQ plumbing can range from tiny pipes to full-blown service-oriented brokers. The messaging industry calls this *intermediation*, meaning that the stuff in the middle deals with either side. In ØMQ, we call these proxies, queues, forwarders, devices, or brokers, depending on the context.

This pattern is extremely common in the real world and is why our societies and economies are filled with intermediaries who have no other real function than to reduce the complexity and scaling costs of larger networks. Real-world intermediaries are typically called wholesalers, distributors, managers, and so on.

The Dynamic Discovery Problem

One of the problems you will hit as you design larger distributed architectures is *discovery*. That is, how do pieces know about each other? It's especially difficult if pieces come and go; thus, we can call this the “dynamic discovery problem.”

There are several solutions to dynamic discovery. The simplest is to entirely avoid it by hard-coding (or configuring) the network architecture so discovery is done by hand.

→ That is, when you add a new piece, you reconfigure the network to know about it.

In practice, this leads to increasingly fragile and unwieldy architectures. Let's say you have one publisher and a hundred subscribers. You connect each subscriber to the publisher by configuring a publisher endpoint in each subscriber. That's easy (Figure 2-4). Subscribers are dynamic; the publisher is static. Now say you add more publishers. Suddenly, it's not so easy any more. If you continue to connect each subscriber to each publisher, the cost of avoiding dynamic discovery gets higher and higher.

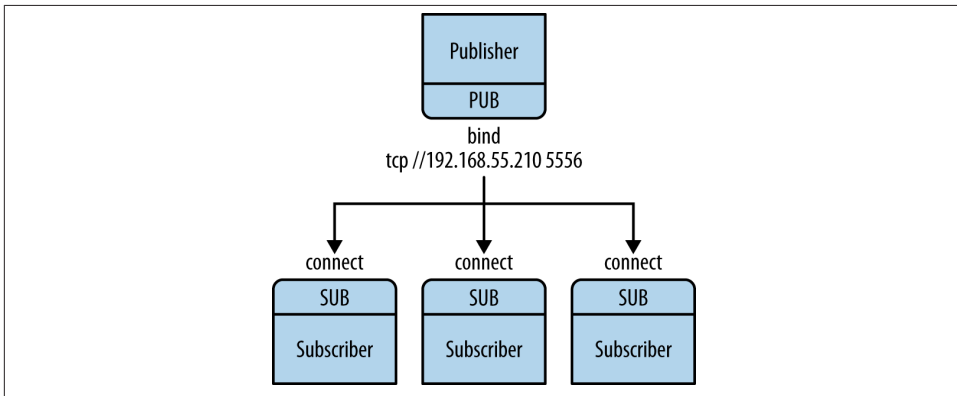


Figure 2-4. Small-scale pub-sub network

There are quite a few solutions to this problem, but the very simplest is to add an intermediary; that is, a static point in the network to which all other nodes connect. In classic messaging, this is the job of the message broker. ØMQ doesn't come with a message broker as such, but it lets us build intermediaries quite easily.

You might wonder, if all networks eventually get large enough to need intermediaries, why don't we simply have a message broker in place for all applications? For beginners, it's a fair compromise. Just always use a star topology, forget about performance, and things will usually work. However, message brokers are greedy things; in their role as central intermediaries, they become too complex, too stateful, and eventually a problem.

It's better to think of intermediaries as simple stateless message switches. The best analogy is an HTTP proxy; it's there, but doesn't have any special role. Adding a pub-sub proxy solves the dynamic discovery problem in our example. We set the proxy in the "middle" of the network (Figure 2-5). The proxy opens an XSUB socket and an XPUB socket, and binds each to well-known IP addresses and ports. Then, all other processes

connect to the proxy, instead of to each other. It becomes trivial to add more subscribers or publishers.

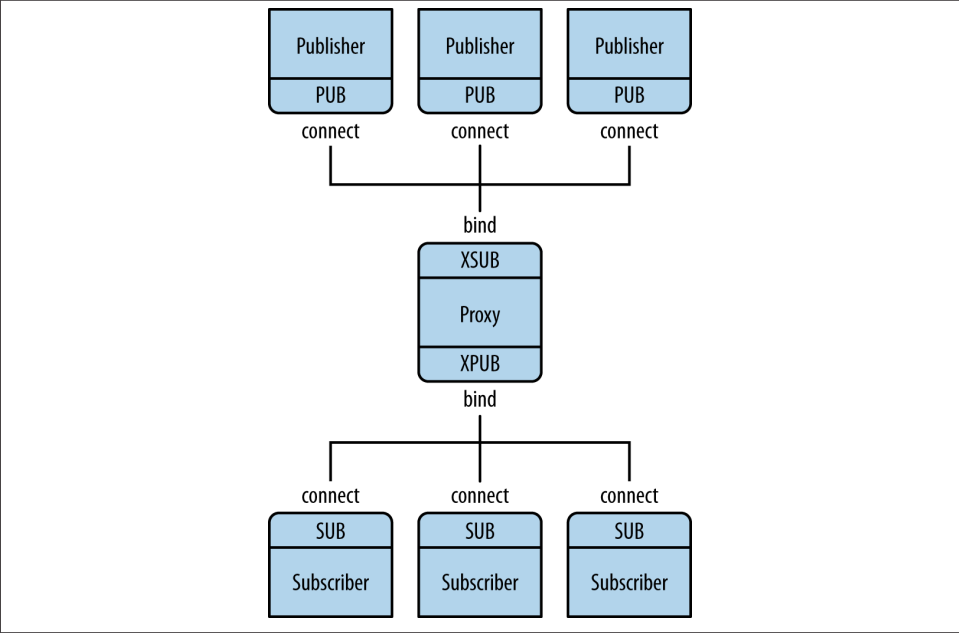


Figure 2-5. Pub-sub network with a proxy

We need XPUB and XSUB sockets because ØMQ does subscription forwarding: SUB sockets actually send subscriptions to PUB sockets as special messages. The proxy has to forward these as well, by **reading them from the XPUB socket and writing them to the XSUB socket**. This is the main use case for XSUB and XPUB (Figure 2-6).

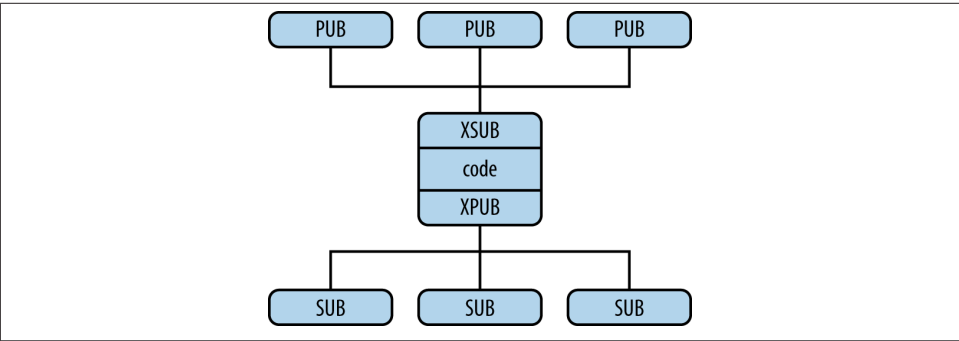


Figure 2-6. Extended publish-subscribe

Shared Queue (DEALER and ROUTER Sockets)

In the “Hello World” client/server application, we have one client that talks to one service. However, in real cases we usually need to allow multiple services as well as multiple clients. This lets us scale up the power of the service (many threads or processes or nodes rather than just one). The only constraint is that services must be stateless, with all state being in the request or in some shared storage, such as a database.

There are two ways to connect multiple clients to multiple servers. ^{Solution: 1.} The brute-force way is to connect each client socket to multiple service endpoints. One client socket can connect to multiple service sockets, and the REQ socket will then distribute requests among these services. Let's say you connect a client socket to three service endpoints: A, B, and C. The client makes requests R1, R2, R3, R4. R1 and R4 go to service A, R2 goes to B, and R3 goes to service C (Figure 2-7).

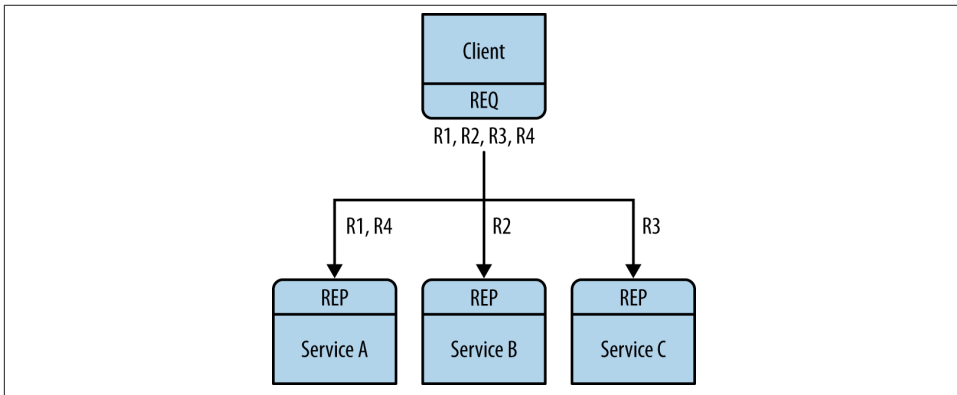


Figure 2-7. Request distribution

This design lets you add more clients cheaply. You can also add more services. Each client will distribute its requests to the services. But each client has to know the service topology. (If you have 100 clients and then you decide to add three more services, you need to reconfigure and restart all 100 clients in order for the clients to know about the three new services.) 13.1

That's clearly not the kind of thing we want to be doing at 3 a.m. when our supercomputing cluster has run out of resources and we desperately need to add a couple of hundred of new service nodes. Too many static pieces are like liquid concrete: knowledge is distributed, and the more static pieces you have, the more effort it is to change the topology. What we want is something sitting in between clients and services that centralizes all knowledge of the topology. Ideally, we should be able to add and remove services or clients at any time without touching any other part of the topology.

Solution 2.

So, we'll write a little message queuing broker that gives us this flexibility. The broker binds to two endpoints, a frontend for clients and a backend for services. It then uses `zmq_poll()` to monitor these two sockets for activity, and when it has some, it shuttles messages between its two sockets. It doesn't actually manage any queues explicitly—ØMQ does that automatically on each socket.

When you use REQ to talk to REP, you get a strictly synchronous request-reply dialog. The client sends a request. The service reads the request and sends a reply. The client then reads the reply. If either the client or the service tries to do anything else (e.g., sending two requests in a row without waiting for a response), it will get an error.

But our broker has to be non-blocking. Obviously, we can use `zmq_poll()` to wait for activity on either socket, but we can't use REP and REQ.

Luckily, there are two sockets called DEALER and ROUTER that let you do non-blocking request-response. You'll see in Chapter 3 how DEALER and ROUTER sockets let you build all kinds of asynchronous request-reply flows. For now, we're just going to see how DEALER and ROUTER let us extend REQ-REP across an intermediary—that is, our little broker.

In this simple extended request-reply pattern, REQ talks to ROUTER and DEALER talks to REP. In between the DEALER and ROUTER, we have to have code (like our broker) that pulls messages off one socket and shoves them onto the other (Figure 2-8).

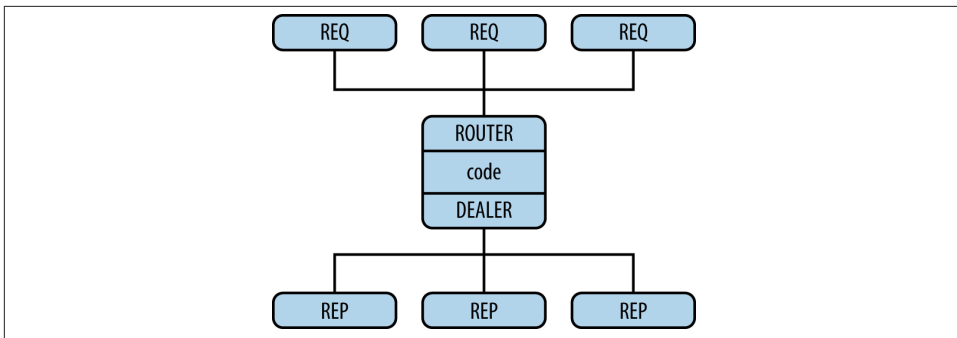


Figure 2-8. Extended request-reply

The request-reply broker binds to two endpoints, one for clients to connect to (the frontend socket) and one for workers to connect to (the backend). To test this broker, you will want to change your workers so they connect to the backend socket. **Example 2-3** is a client that shows what I mean.

Example 2-3. Request-reply client (rrclient.c)

```
//  
// Hello World client  
// Connects REQ socket to tcp://localhost:5559  
// Sends "Hello" to server, expects "World" back  
//  
#include "zhelpers.h"  
  
int main (void)  
{  
    void *context = zmq_ctx_new ();  
  
    // Socket to talk to server  
    void *requester = zmq_socket (context, ZMQ_REQ);  
    zmq_connect (requester, "tcp://localhost:5559");  
  
    int request_nbr;  
    for (request_nbr = 0; request_nbr != 10; request_nbr++) {  
        s_send (requester, "Hello");  
        char *string = s_recv (requester);  
        printf ("Received reply %d [%s]\n", request_nbr, string);  
        free (string);  
    }  
    zmq_close (requester);  
    zmq_ctx_destroy (context);  
    return 0;  
}
```

The code for the worker is in **Example 2-4**.

Example 2-4. Request-reply worker (rrworker.c)

```
//  
// Hello World worker  
// Connects REP socket to tcp://*:5560  
// Expects "Hello" from client, replies with "World"  
//  
#include "zhelpers.h"  
  
int main (void)  
{  
    void *context = zmq_ctx_new ();  
  
    // Socket to talk to clients  
    void *responder = zmq_socket (context, ZMQ_REP);  
    zmq_connect (responder, "tcp://localhost:5560");
```

```

while (1) {
    // Wait for next request from client
    char *string = s_recv (responder);
    printf ("Received request: [%s]\n", string);
    free (string);

    // Do some 'work'
    sleep (1);

    // Send reply back to client
    s_send (responder, "World");
}
// We never get here, but clean up anyhow
zmq_close (responder);
zmq_ctx_destroy (context);
return 0;
}

```

And finally, the code for the broker, which properly handles multipart messages, is in [Example 2-5](#).

Example 2-5. Request-reply broker (rrbroker.c)

```

//
// Simple request-reply broker
//
#include "zhelpers.h"

int main (void)
{
    // Prepare our context and sockets
    void *context = zmq_ctx_new ();
    void *frontend = zmq_socket (context, ZMQ_ROUTER);
    void *backend = zmq_socket (context, ZMQ_DEALER);
    zmq_bind (frontend, "tcp://*:5559");
    zmq_bind (backend, "tcp://*:5560");

    // Initialize poll set
    zmq_pollitem_t items [] = {
        { frontend, 0, ZMQ_POLLIN, 0 },
        { backend, 0, ZMQ_POLLIN, 0 }
    };

    // Switch messages between sockets
    while (1) {
        zmq_msg_t message;
        int more; // Multipart detection

        zmq_poll (items, 2, -1);
        if (items [0].revents & ZMQ_POLLIN) {
            while (1) {
                // Process all parts of the message

```

```

        zmq_msg_init (&message);
        zmq_msg_recv (&message, frontend, 0);
        size_t more_size = sizeof (more);
        zmq_getsockopt (frontend, ZMQ_RCVMORE, &more, &more_size);
        zmq_msg_send (&message, backend, more? ZMQ_SNDMORE: 0);
        zmq_msg_close (&message);
        if (!more)
            break; // Last message part
    }
}
if (items [1].revents & ZMQ_POLLIN) {
    while (1) {
        // Process all parts of the message
        zmq_msg_init (&message);
        zmq_msg_recv (&message, backend, 0);
        size_t more_size = sizeof (more);
        zmq_getsockopt (backend, ZMQ_RCVMORE, &more, &more_size);
        zmq_msg_send (&message, frontend, more? ZMQ_SNDMORE: 0);
        zmq_msg_close (&message);
        if (!more)
            break; // Last message part
    }
}
}
// We never get here, but clean up anyhow
zmq_close (frontend);
zmq_close (backend);
zmq_ctx_destroy (context);
return 0;
}

```

Using a request-reply broker makes your client/server architectures easier to scale, because clients don't see workers and workers don't see clients. The only static node is the broker in the middle (Figure 2-9).

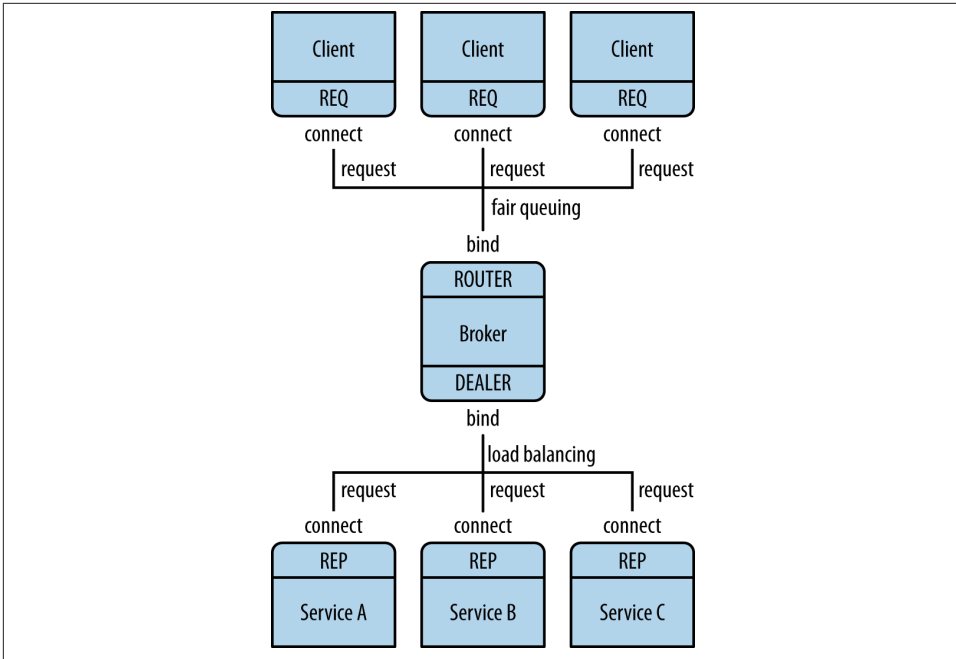


Figure 2-9. Request-reply broker

ØMQ's Built-in Proxy Function

It turns out that the core loop in the previous section's *rrbroker* is very useful, and reusable. It lets us build pub-sub forwarders and shared queues and other little intermediaries with very little effort. ØMQ wraps this up in a single method, `zmq_proxy()`:

```
zmq_proxy (frontend, backend, capture);
```

The two sockets (or three if we want to capture data) must be properly connected, bound, and configured. When we call the `zmq_proxy()` method, it's exactly like starting the main loop of *rrbroker*. Let's rewrite the request-reply broker to call `zmq_proxy()`, and re-badge this as an expensive-sounding "message queue" (people have charged houses for code that did less). Example 2-6 shows the result.

Example 2-6. Message queue broker (*msgqueue.c*)

```
//
// Simple message queuing broker
// Same as request-reply broker but using QUEUE device
//
#include "zhelpers.h"

int main (void)
{
```

```

void *context = zmq_ctx_new ();

// Socket facing clients
void *frontend = zmq_socket (context, ZMQ_ROUTER);
int rc = zmq_bind (frontend, "tcp://*:5559");
assert (rc == 0);

// Socket facing services
void *backend = zmq_socket (context, ZMQ_DEALER);
rc = zmq_bind (backend, "tcp://*:5560");
assert (rc == 0);

// Start the proxy
zmq_proxy (frontend, backend, NULL);

// We never get here...
zmq_close (frontend);
zmq_close (backend);
zmq_ctx_destroy (context);
return 0;
}

```

If you're like most ØMQ users, at this stage you're starting to think, "What kind of evil stuff can I do if I plug random socket types into the proxy?" The short answer is: try it and work out what is happening. In practice, you would usually stick to ROUTER/DEALER, XSUB/XPUB, or PULL/PUSH.

Transport Bridging

bridge 的定义

A frequent request from ØMQ users is, "How do I connect my ØMQ network with technology X?" where X is some other networking or messaging technology. The simple answer is to build a "bridge." A bridge is a small application that speaks one protocol at one socket, and converts to/from a second protocol at another socket. A protocol interpreter, if you like. A common bridging problem in ØMQ is to bridge two transports or networks.

As an example, we're going to write a little proxy (Example 2-7) that sits in between a publisher and a set of subscribers, bridging two networks. The frontend socket (SUB) faces the internal network where the weather server is sitting, and the backend (PUB) faces subscribers on the external network. It subscribes to the weather service on the frontend socket, and republishes its data on the backend socket (Figure 2-10).

Example 2-7. Weather update proxy (wuproxy.c)

```

//
// Weather proxy device
//
#include "zhelpers.h"

int main (void)

```



```

{
    void *context = zmq_ctx_new ();

    // This is where the weather server sits
    void *frontend = zmq_socket (context, ZMQ_XSUB);
    zmq_connect (frontend, "tcp://192.168.55.210:5556");

    // This is our public endpoint for subscribers
    void *backend = zmq_socket (context, ZMQ_XPUB);
    zmq_bind (backend, "tcp://10.1.1.0:8100");

    // Run the proxy until the user interrupts us
    zmq_proxy (frontend, backend, NULL);

    zmq_close (frontend);
    zmq_close (backend);
    zmq_ctx_destroy (context);
    return 0;
}

```

在不同網路

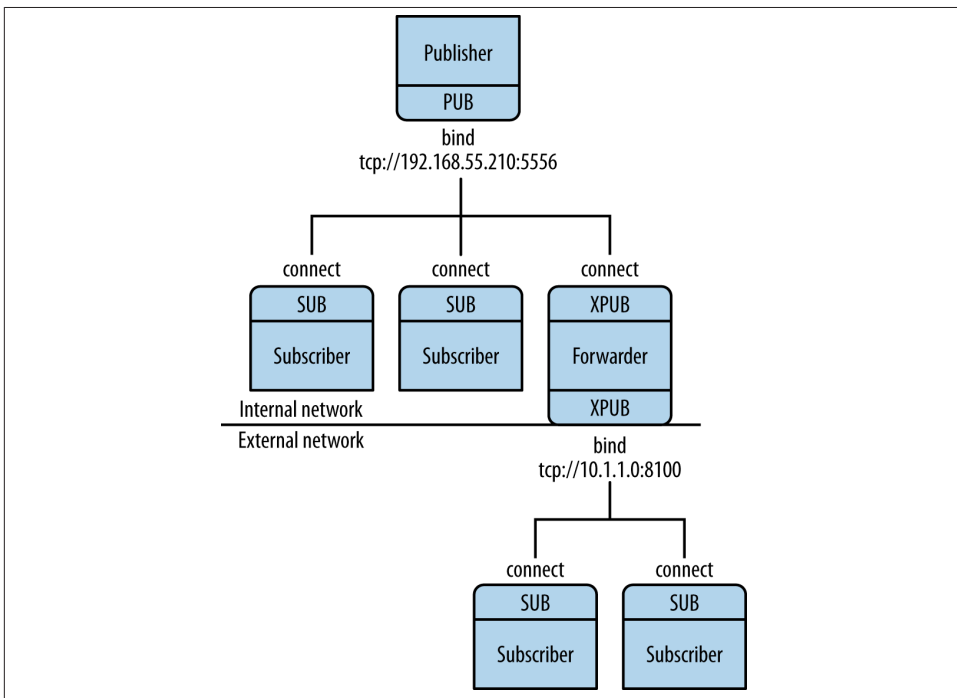


Figure 2-10. Pub-sub forwarder proxy

It looks very similar to the earlier proxy example, but the key part is that the frontend and backend sockets are on two different networks. We can use this model, for example, to connect a multicast network (pgm transport) to a TCP publisher.

Handling Errors and ETERM

彈性復原

ØMQ's error handling philosophy is a mix of fail fast and resilience. Processes, we believe, should be as vulnerable as possible to internal errors, and as robust as possible against external attacks and errors. To give an analogy, a living cell will self-destruct if it detects a single internal error, yet it will resist attack from the outside by all means possible.

Assertions, which pepper the ØMQ code, are absolutely vital to robust code; they just have to be on the right side of the cellular wall. And there should be such a wall. If it is unclear whether a fault is internal or external, that is a design flaw to be fixed. In C/C++, assertions stop the application immediately with an error. In other languages, you may get exceptions or halts.

When ØMQ detects an external fault, it returns an error to the calling code. In some rare cases, it drops messages silently if there is no obvious strategy for recovering from the error.

In most of the C examples we've seen so far, there's been no error handling. *Real code should do error handling on every single ØMQ call.* If you're using a language binding other than C, the binding may handle errors for you. In C, you do need to do this yourself. There are some simple rules, starting with POSIX conventions:

- Methods that create objects return NULL if they fail.
- Methods that process data may return the number of bytes processed, or -1 on an error or failure.
- Other methods return 0 on success and -1 on an error or failure.
- The error code is provided in `errno` or `zmq_errno()`.
- Descriptive error text for logging is provided by `zmq_strerror()`.

For example:

```
void *context = zmq_ctx_new ();
assert (context);
void *socket = zmq_socket (context, ZMQ_REP);
assert (socket);
int rc = zmq_bind (socket, "tcp://*:5555");
if (rc != 0) {
    printf ("E: bind failed: %s\n", strerror (errno));
    return -1;
}
```

There are two main exceptional conditions that you may want to handle as nonfatal:

- When a thread calls `zmq_msg_recv()` with the `ZMQ_DONTWAIT` option and there is no waiting data, ØMQ will return `-1` and set `errno` to `EAGAIN`.
- When a thread calls `zmq_ctx_destroy()` and other threads are doing blocking work, the `zmq_ctx_destroy()` call closes the context and all blocking calls exit with `-1` and `errno` set to `ETERM`.

In C/C++, asserts can be removed entirely in optimized code, so don't make the mistake of wrapping the whole ØMQ call in an `assert()`. It looks neat; then the optimizer removes all the asserts and the calls you want to make, and your application breaks in impressive ways.

Let's see how to shut down a process cleanly. We'll take the parallel pipeline example from the previous section. If we've started a whole lot of workers in the background, we now want to kill them when the batch is finished. Let's do this by sending a kill message to the workers. The best place to do this is the sink, because it really knows when the batch is done.

How do we connect the sink to the workers? (为了 kill workers)
The PUSH/PULL sockets are one-way only. The standard ØMQ answer is: create a new socket flow for each type of problem you need to solve. We'll use a publish-subscribe model to send kill messages to the workers:

- The sink creates a PUB socket on a new endpoint.
- Workers bind their input sockets to this endpoint.
- When the sink detects the end of the batch, it sends a kill to its PUB socket.
- When a worker detects this kill message, it exits.

It doesn't take much new code in the sink:

```
void *control = zmq_socket (context, ZMQ_PUB);
zmq_bind (control, "tcp://*:5559");
...
// Send kill signal to workers
zmq_msg_init_data (&message, "KILL", 5);
zmq_msg_send (control, &message, 0);
zmq_msg_close (&message);
```

Figure 2-11 illustrates the resulting configuration.

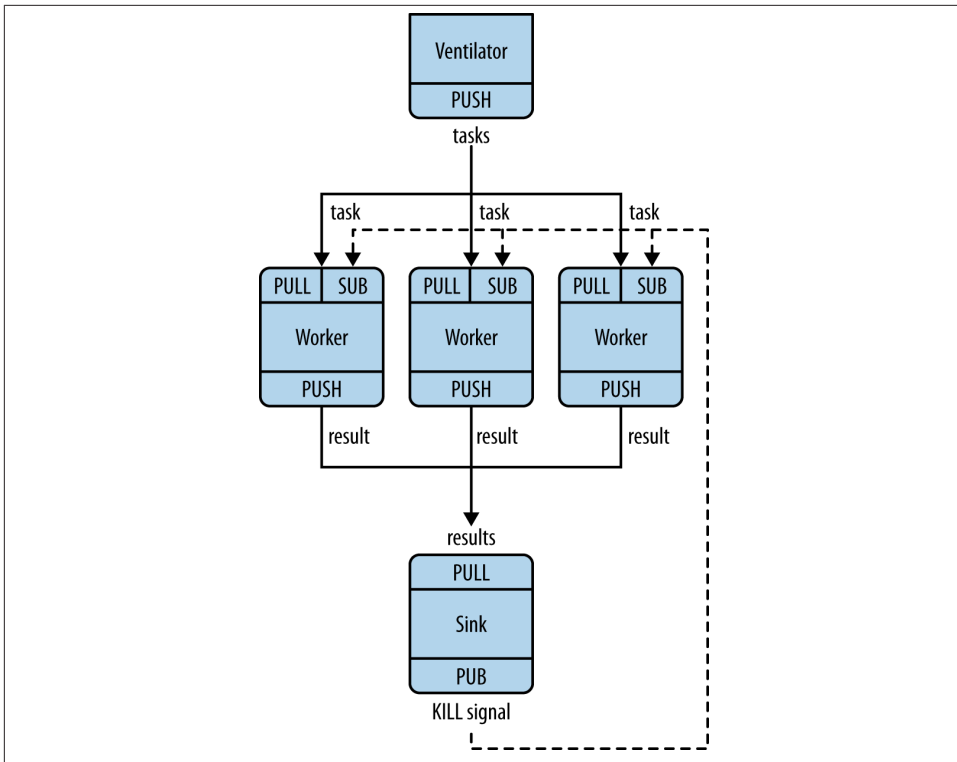


Figure 2-11. Parallel pipeline with kill signaling

Example 2-8 contains the code for the worker process, which manages two sockets (a PULL socket getting tasks, and a SUB socket getting control commands) using the `zmq_poll()` technique we saw earlier.

Example 2-8. Parallel task worker with kill signaling (taskwork2.c)

```
//
// Task worker - design 2
// Adds pub-sub flow to receive and respond to kill signal
//
#include "zhelpers.h"

int main (void)
{
    void *context = zmq_ctx_new ();

    // Socket to receive messages on
    void *receiver = zmq_socket (context, ZMQ_PULL);
    zmq_connect (receiver, "tcp://localhost:5557");

    // Socket to send messages to
```

```

void *sender = zmq_socket (context, ZMQ_PUSH);
zmq_connect (sender, "tcp://localhost:5558");

// Socket for control input
void *controller = zmq_socket (context, ZMQ_SUB);
zmq_connect (controller, "tcp://localhost:5559");
zmq_setsockopt (controller, ZMQ_SUBSCRIBE, "", 0);

// Process messages from receiver and controller
zmq_pollitem_t items [] = {
    { receiver, 0, ZMQ_POLLIN, 0 },
    { controller, 0, ZMQ_POLLIN, 0 }
};

// Process messages from both sockets
while (1) {
    zmq_msg_t message;
    zmq_poll (items, 2, -1);
    if (items [0].revents & ZMQ_POLLIN) {
        zmq_msg_init (&message);
        zmq_msg_recv (&message, receiver, 0);

        // Do the work
        s_sleep (atoi ((char *) zmq_msg_data (&message)));

        // Send results to sink
        zmq_msg_init (&message);
        zmq_msg_send (&message, sender, 0);

        // Simple progress indicator for the viewer
        printf (".");
        fflush (stdout);

        zmq_msg_close (&message);
    }
    // Any waiting controller command acts as 'KILL'
    if (items [1].revents & ZMQ_POLLIN)
        break; // Exit loop
}

// Finished
zmq_close (receiver);
zmq_close (sender);
zmq_close (controller);
zmq_ctx_destroy (context);
return 0;
}

```

Example 2-9 shows the modified sink application. When it's finished collecting results, it broadcasts a kill message to all workers.

Example 2-9. Parallel task sink with kill signaling (tasksink2.c)

```
//
// Task sink - design 2
// Adds pub-sub flow to send kill signal to workers
//
#include "zhelpers.h"

int main (void)
{
    void *context = zmq_ctx_new ();

    // Socket to receive messages on
    void *receiver = zmq_socket (context, ZMQ_PULL);
    zmq_bind (receiver, "tcp://*:5558");

    // Socket for worker control
    void *controller = zmq_socket (context, ZMQ_PUB);
    zmq_bind (controller, "tcp://*:5559");

    // Wait for start of batch
    char *string = s_recv (receiver);
    free (string);

    // Start our clock now
    int64_t start_time = s_clock ();

    // Process 100 confirmations
    int task_nbr;
    for (task_nbr = 0; task_nbr < 100; task_nbr++) {
        char *string = s_recv (receiver);
        free (string);
        if ((task_nbr / 10) * 10 == task_nbr)
            printf (":");
        else
            printf (".");
        fflush (stdout);
    }
    printf ("Total elapsed time: %d msec\n",
        (int) (s_clock () - start_time));

    // Send kill signal to workers
    s_send (controller, "KILL");

    // Finished
    sleep (1);           // Give OMQ time to deliver

    zmq_close (receiver);
    zmq_close (controller);
    zmq_ctx_destroy (context);
    return 0;
}
```

Handling Interrupt Signals

Realistic applications need to shut down cleanly when interrupted with Ctrl-C or another signal, such as SIGTERM. By default, **these simply kill the process, meaning messages won't be flushed, files won't be closed cleanly, and so on.**

Example 2-10 shows how we handle signals in various languages.

Example 2-10. Handling Ctrl-C cleanly (interrupt.c)

```
//
// Shows how to handle Ctrl-C
//
#include <zmq.h>
#include <stdio.h>
#include <signal.h>

// -----
// Signal handling
//
// Call s_catch_signals() in your application at startup, and then exit
// your main loop if s_interrupted is ever 1. Works especially well with
// zmq_poll.

static int s_interrupted = 0;
static void s_signal_handler (int signal_value)
{
    s_interrupted = 1;
}

static void s_catch_signals (void)
{
    struct sigaction action;
    action.sa_handler = s_signal_handler;
    action.sa_flags = 0;
    sigemptyset (&action.sa_mask);
    sigaction (SIGINT, &action, NULL);
    sigaction (SIGTERM, &action, NULL);
}

int main (void)
{
    void *context = zmq_ctx_new ();
    void *socket = zmq_socket (context, ZMQ_REP);
    zmq_bind (socket, "tcp://*:5555");

    s_catch_signals ();
    while (1) {
        // Blocking read will exit on a signal
        zmq_msg_t message;
        zmq_msg_init (&message);
        zmq_msg_recv (&message, socket, 0);
    }
}
```

```

        if (s_interrupted) {
            printf ("W: interrupt received, killing server...\n");
            break;
        }
    }
    zmq_close (socket);
    zmq_ctx_destroy (context);
    return 0;
}

```

The program provides `s_catch_signals()`, which traps Ctrl-C (SIGINT) and SIGTERM. When either of these signals arrive, the `s_catch_signals()` handler sets the global variable `s_interrupted`. Thanks to your signal handler, your application will not die automatically. Instead, you have a chance to clean up and exit gracefully. You have to now explicitly check for an interrupt and handle it properly. Do this by calling `s_catch_signals()` (copy this from *interrupt.c*) at the start of your main code. This sets up the signal handling. The interrupt will affect ØMQ calls as follows:

- If your code is blocking in `zmq_msg_recv()`, `zmq_poll()`, or `zmq_msg_send()`, when a signal arrives, the call will return with EINTR.
- Wrappers like `s_recv()` return NULL if they are interrupted.

So, check for an EINTR return code, a NULL return, and/or `s_interrupted`.

Here is a typical code fragment:

```

s_catch_signals ();
client = zmq_socket (...);
while (!s_interrupted) {
    char *message = s_recv (client);
    if (!message)
        break;          // Ctrl-C used
}
zmq_close (client);

```

If you call `s_catch_signals()` and don't test for interrupts, your application will become immune to Ctrl-C and SIGTERM, which may be useful but is usually not.

Detecting Memory Leaks 教我們用其它套件管理 memory

Any long-running application has to manage memory correctly, or eventually it'll use up all available memory and crash. If you use a language that handles this automatically for you, congratulations. If you program in C or C++ or any other language where you're responsible for memory management, here's a short tutorial on using `valgrind`, which, among other things, will report on any leaks your programs have:

- To install valgrind, such as on Ubuntu or Debian, issue:

```
sudo apt-get install valgrind
```

- By default, ØMQ will cause valgrind to complain a lot. To remove these warnings, create a file called *valgrind.suppress* that contains this:

```
{
  <socketcall_sendto>
  Memcheck:Param
  socketcall.sendto(msg)
  fun:send
  ...
}
{
  <socketcall_sendto>
  Memcheck:Param
  socketcall.send(msg)
  fun:send
  ...
}
```

- Fix your applications to exit cleanly after Ctrl-C. For any application that exits by itself, that's not needed, but for long-running applications, this is essential. Otherwise, valgrind will complain about all currently allocated memory.
- Build your application with *-DDEBUG*, if it's not your default setting. That ensures valgrind can tell you exactly where memory is being leaked.
- Finally, run valgrind as follows (all on one line)

```
valgrind --tool=memcheck --leak-check=full --suppressions=valgrind.suppress
someprog
```

After fixing any errors it reports, you should get the pleasant message:

```
==30536== ERROR SUMMARY: 0 errors from 0 contexts...
```

Multithreading with ØMQ

ØMQ is perhaps the nicest way ever to write multithreaded (MT) applications. Whereas ØMQ sockets require some readjustment if you are used to traditional sockets, ØMQ multithreading will take everything you know about writing MT applications, throw it into a heap in the garden, pour gasoline over it, and set it alight. It's a rare book that deserves burning, but most books on concurrent programming do.

To make utterly perfect MT programs (and I mean that literally), *we don't need mutexes, locks, or any other form of inter-thread communication except messages sent across ØMQ sockets.*

By “perfect” MT programs, I mean code that’s easy to write and understand, that works with the same design approach in any programming language and on any operating system, and that scales across any number of CPUs with zero wait states and no point of diminishing returns.

If you’ve spent years learning tricks to make your MT code work at all, let alone rapidly, with locks and semaphores and critical sections, you will be disgusted when you realize it was all for nothing. If there’s one lesson we’ve learned from 30+ years of concurrent programming, it is: *just don’t share state*. It’s like two drunkards trying to share a beer. It doesn’t matter if they’re good buddies. Sooner or later, they’re going to get into a fight. And the more drunkards you add to the table, the more they fight each other over the beer. The tragic majority of MT applications look like drunken bar fights.

The list of weird problems that you need to fight as you write classic shared-state MT code would be hilarious if it didn’t translate directly into stress and risk, as code that seems to work suddenly fails under pressure. A few years ago, a large firm with world-beating experience in buggy code released its list of “11 Likely Problems in Your Multithreaded Code,” which covers forgotten synchronization, incorrect granularity, read and write tearing, lock-free reordering, lock convoys, the two-step dance, and priority inversion.

Yeah, we counted seven problems, not eleven. That’s not the point, though. The point is, do you really want code running the power grid or stock market to start getting two-step lock convoys at 3 p.m. on a busy Thursday? Who cares what the terms actually mean? This is not what turned us on to programming, fighting ever more complex side effects with ever more complex hacks.

Some widely used models, despite being the basis for entire industries, are fundamentally broken, and shared state concurrency is one of them. Code that wants to scale without limit does it like the Internet does, by sending messages and sharing nothing except a common contempt for broken programming models.

中不要

You should follow some rules to write happy multithreaded code with ØMQ:

- You must not access the same data from multiple threads. Using classic MT techniques like mutexes is an anti-pattern in ØMQ applications. The only exception to this is a ØMQ context object, which is threadsafe.
- You must create a ØMQ context for your process, and pass that to all threads that you want to connect via inproc sockets.
- You may treat threads as separate tasks with their own context, but these threads cannot communicate over inproc. However, they will be easier to break into stand-alone processes afterwards.
- You must not share ØMQ sockets between threads. ØMQ sockets are not thread-safe. Technically it’s possible to do this, but it demands semaphores, locks, or mu-

texes. This will make your application slow and fragile. The only place where it's remotely sane to share sockets between threads is in language bindings that need to do magic like garbage collection on sockets.

If you need to start more than one proxy in an application, for example, you will want to run each in its own thread. It is easy to make the error of creating the proxy frontend and backend sockets in one thread, and then passing the sockets to the proxy in another thread. This may appear to work but will fail randomly. Remember: *do not use or close sockets except in the thread that created them.*

If you follow these rules, you can quite easily split threads into separate processes when you need to. Application logic can sit in threads, processes or nodes: whatever your scale needs.

ØMQ uses native OS threads rather than virtual “green” threads. The advantages are that you don't need to learn any new threading API, and that ØMQ threads map cleanly to your operating system. You can use standard tools like Intel's ThreadChecker to see what your application is doing. The disadvantages are that your code, when it for instance starts new threads, won't be portable, and that if you have a huge number of threads (in the thousands), some operating systems will get stressed.

Let's see how this works in practice. We'll turn our old Hello World server into something more capable. The original server ran in a single thread. If the work per request is low, that's fine: one ØMQ thread can run at full speed on a CPU core, with no waits, doing an awful lot of work. But realistic servers have to do nontrivial work per request. A single core may not be enough when 10,000 clients hit the server all at once. So a realistic server must start multiple worker threads. It then accepts requests as fast as it can and distributes these to its worker threads. The worker threads grind through the work and eventually send their replies back.

You can, of course, do all this using a proxy broker and external worker processes, but often it's easier to start one process that gobbles up 16 cores than 16 processes, each gobbling up one core. Further, running workers as threads will cut out a network hop, latency, and network traffic.

The MT version of the Hello World service in [Example 2-11](#) basically collapses the broker and workers into a single process. We use pthreads because it's the most widespread standard for multithreading.

Example 2-11. Multithreaded service (mtserver.c)

```
//  
// Multithreaded Hello World server  
//  
#include "zhelpers.h"  
#include <pthread.h>
```

```

static void *
worker_routine (void *context) {
    // Socket to talk to dispatcher
    void *receiver = zmq_socket (context, ZMQ_REP);
    zmq_connect (receiver, "inproc://workers");

    while (1) {
        char *string = s_recv (receiver);
        printf ("Received request: [%s]\n", string);
        free (string);
        // Do some 'work'
        sleep (1);
        // Send reply back to client
        s_send (receiver, "World");
    }
    zmq_close (receiver);
    return NULL;
}

int main (void)
{
    void *context = zmq_ctx_new ();

    // Socket to talk to clients
    void *clients = zmq_socket (context, ZMQ_ROUTER);
    zmq_bind (clients, "tcp://*:5555");

    // Socket to talk to workers
    void *workers = zmq_socket (context, ZMQ_DEALER);
    zmq_bind (workers, "inproc://workers");

    // Launch pool of worker threads
    int thread_nbr;
    for (thread_nbr = 0; thread_nbr < 5; thread_nbr++) {
        pthread_t worker;
        pthread_create (&worker, NULL, worker_routine, context);
    }
    // Connect work threads to client threads via a queue proxy
    zmq_proxy (clients, workers, NULL);

    // We never get here, but clean up anyhow
    zmq_close (clients);
    zmq_close (workers);
    zmq_ctx_destroy (context);
    return 0;
}

```

All the code should be recognizable to you by now. Here's how it works:

- The server starts a set of worker threads. Each worker thread creates a REP socket and then processes requests on this socket. Worker threads are just like single-

threaded servers. The only differences are the transport (`inproc` instead of `tcp`) and the bind-connect direction.

- The server creates a ROUTER socket to talk to clients and binds this to its external interface (over `tcp`).
- The server creates a DEALER socket to talk to the workers and binds this to its internal interface (over `inproc`).
- The server starts a proxy that connects the two sockets. The proxy pulls incoming requests fairly from all clients and distributes those out to workers. It also routes replies back to their origin.

Note that creating threads is not portable in most programming languages. The POSIX library is `pthread`s, but on Windows you have to use a different API. In our example, the `pthread_create()` call starts up a new thread running the `worker_routine()` function we defined. We'll see in [Chapter 3](#) how to wrap this in a portable API.

Here, the “work” is just a one-second pause. We could do anything in the workers, though, including talking to other nodes. Figure 2-12 shows what the MT server looks like in terms of ØMQ sockets and nodes. Note how the request-reply chain is REQ-ROUTER-queue-DEALER-REP.

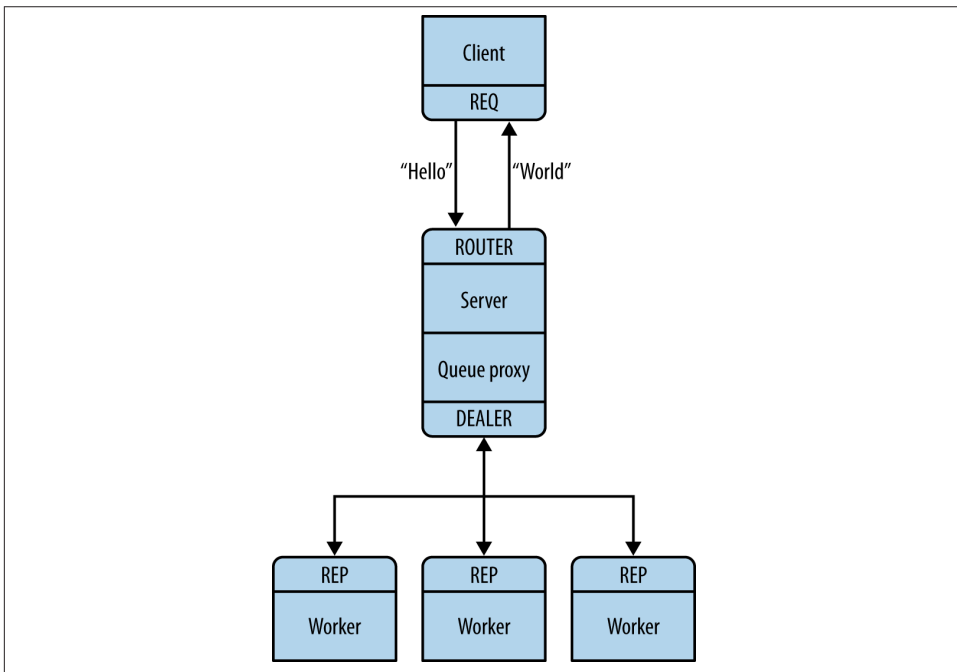


Figure 2-12. Multithreaded server

Signaling Between Threads (PAIR Sockets)

When you start making multithreaded applications with ØMQ, you'll encounter the question of how to coordinate your threads. Though you might be tempted to insert “sleep” statements, or use multithreading techniques such as semaphores or mutexes, *the only mechanism that you should use is ØMQ messages*. Remember the story of the drunkards and the beer.

Let's make three threads that signal each other when they are ready (Figure 2-13). In Example 2-12, we use PAIR sockets over the inproc transport.

Example 2-12. Multithreaded relay (mtrelay.c)

```
//
// Multithreaded relay
//
#include "zhelpers.h"
#include <pthread.h>

static void *
step1 (void *context) {
    // Connect to step2 and tell it we're ready
    void *xmitter = zmq_socket (context, ZMQ_PAIR);
    zmq_connect (xmitter, "inproc://step2");
    printf ("Step 1 ready, signaling step 2\n");
    s_send (xmitter, "READY");
    zmq_close (xmitter);

    return NULL;
}

static void *
step2 (void *context) {
    // Bind inproc socket before starting step1
    void *receiver = zmq_socket (context, ZMQ_PAIR);
    zmq_bind (receiver, "inproc://step2");
    pthread_t thread;
    pthread_create (&thread, NULL, step1, context);

    // Wait for signal and pass it on
    char *string = s_recv (receiver);
    free (string);
    zmq_close (receiver);

    // Connect to step3 and tell it we're ready
    void *xmitter = zmq_socket (context, ZMQ_PAIR);
    zmq_connect (xmitter, "inproc://step3");
    printf ("Step 2 ready, signaling step 3\n");
    s_send (xmitter, "READY");
    zmq_close (xmitter);
}
```

```

    return NULL;
}

int main (void)
{
    void *context = zmq_ctx_new ();

    // Bind inproc socket before starting step2
    void *receiver = zmq_socket (context, ZMQ_PAIR);
    zmq_bind (receiver, "inproc://step3");
    pthread_t thread;
    pthread_create (&thread, NULL, step2, context);

    // Wait for signal
    char *string = s_recv (receiver);
    free (string);
    zmq_close (receiver);

    printf ("Test successful!\n");
    zmq_ctx_destroy (context);
    return 0;
}

```

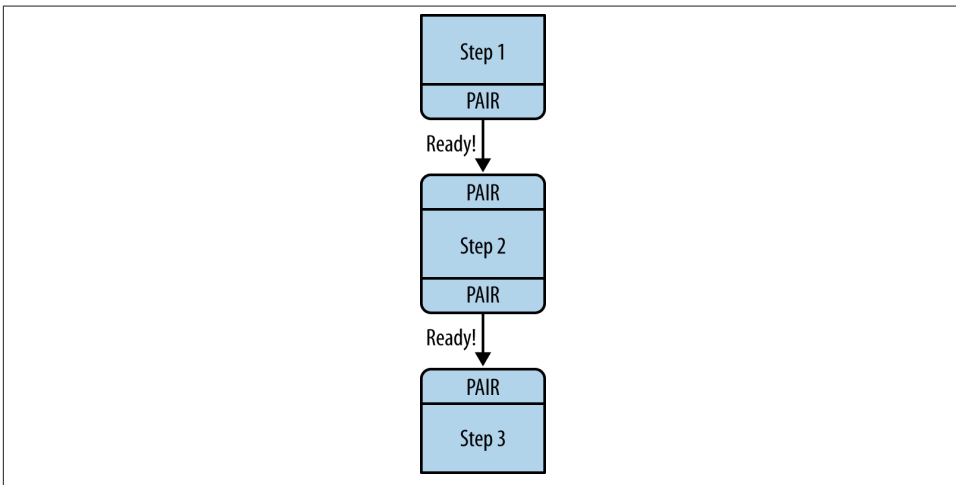


Figure 2-13. The relay race

This is a classic pattern for multithreading with ØMQ:

1. Two threads communicate over inproc, using a shared context.
2. The parent thread creates one socket, binds it to an inproc endpoint, and *then* starts the child thread, passing the context to it.

3. The child thread creates the second socket, connects it to that `inproc` endpoint, and then signals to the parent thread that it's ready.

Note that multithreading code using this pattern is not scalable out to processes. If you use `inproc` and socket pairs, you are building a tightly bound application; i.e., one where your threads are structurally interdependent. Do this when low latency is really vital. The other design pattern is a loosely bound application, where threads have their own context and communicate over `ipc` or `tcp`. You can easily break loosely bound threads into separate processes.

This is the first time we've shown an example using PAIR sockets. Why use PAIR? Other socket combinations might seem to work, but they all have side effects that could interfere with signaling:

- You can use PUSH for the sender and PULL for the receiver. This looks simple and will work, but remember that PUSH will distribute messages to all available receivers. If you by accident start two receivers (e.g., you already have one running and you start a second), you'll "lose" half of your signals. PAIR has the advantage of refusing more than one connection; the pair is *exclusive*.
- You can use DEALER for the sender and ROUTER for the receiver. ROUTER, however, wraps your message in an "envelope," meaning your zero-size signal turns into a multipart message. If you don't care about the data and treat anything as a valid signal, and if you don't read more than once from the socket, that won't matter. If, however, you decide to send real data, you will suddenly find ROUTER providing you with "wrong" messages. DEALER also distributes outgoing messages, giving the same risk as PUSH.
- You can use PUB for the sender and SUB for the receiver. This will correctly deliver your messages exactly as you sent them, and PUB does not distribute as PUSH or DEALER do. However, you need to configure the subscriber with an empty subscription, which is annoying. Worse, the reliability of the PUB-SUB link is timing-dependent, and messages can get lost if the SUB socket is connecting while the PUB socket is sending its messages.

For these reasons, PAIR makes the best choice for coordination between pairs of threads.

Node Coordination

When you want to coordinate nodes, PAIR sockets won't work well any more. This is one of the few areas where the strategies for threads and nodes are different. Principally, nodes come and go whereas threads are static. PAIR sockets do not automatically reconnect if the remote node goes away and comes back.

The second significant difference between threads and nodes is that you typically have a fixed number of threads but a more variable number of nodes. Let's take one of our earlier scenarios (the weather server and clients) and use node coordination to ensure that subscribers don't lose data when starting up.

This is how the application will work:

- The publisher knows in advance how many subscribers it expects. This is just a magic number it gets from somewhere.
- The publisher starts up and waits for all subscribers to connect. This is the node coordination part. Each subscriber subscribes and then tells the publisher it's ready via another socket.
- When the publisher has all subscribers connected, it starts to publish data.

In this case, we'll use a REQ-REP socket flow to synchronize the subscribers and the publisher (Figure 2-14). The code for the publisher is in Example 2-13.

Example 2-13. Synchronized publisher (syncpub.c)

```
//  
// Synchronized publisher  
//  
#include "zhelpers.h"  
  
// We wait for 10 subscribers  
#define SUBSCRIBERS_EXPECTED 10  
  
int main (void)  
{  
    void *context = zmq_ctx_new ();  
  
    // Socket to talk to clients  
    void *publisher = zmq_socket (context, ZMQ_PUB);  
    zmq_bind (publisher, "tcp://*:5561");  
  
    // Socket to receive signals  
    void *syncservice = zmq_socket (context, ZMQ_REP);  
    zmq_bind (syncservice, "tcp://*:5562");  
  
    // Get synchronization from subscribers  
    printf ("Waiting for subscribers\n");  
    int subscribers = 0;  
    while (subscribers < SUBSCRIBERS_EXPECTED) {  
        // - wait for synchronization request  
        char *string = s_recv (syncservice);  
        free (string);  
        // - send synchronization reply  
        s_send (syncservice, "");  
        subscribers++;  
    }  
}
```

```

// Now broadcast exactly 1M updates followed by END
printf ("Broadcasting messages\n");
int update_nbr;
for (update_nbr = 0; update_nbr < 1000000; update_nbr++)
    s_send (publisher, "Rhubarb");

s_send (publisher, "END");

zmq_close (publisher);
zmq_close (syncservice);
zmq_ctx_destroy (context);
return 0;
}

```

REQ, REP 是用來
讓 Publisher 知道 Subscribers
是否連上

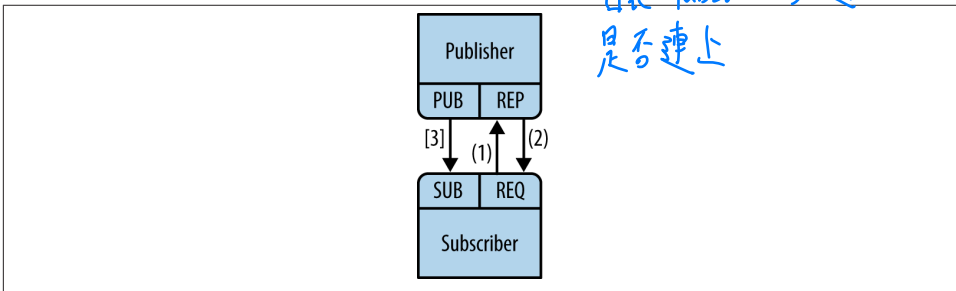


Figure 2-14. Pub-sub synchronization

The code for the subscriber is in [Example 2-14](#).

Example 2-14. Synchronized subscriber (syncsub.c)

```

//
// Synchronized subscriber
//
#include "zhelpers.h"

int main (void)
{
    void *context = zmq_ctx_new ();

    // First, connect our subscriber socket
    void *subscriber = zmq_socket (context, ZMQ_SUB);
    zmq_connect (subscriber, "tcp://localhost:5561");
    zmq_setsockopt (subscriber, ZMQ_SUBSCRIBE, "", 0);

    // zmq is so fast, we need to wait a while...
    sleep (1);

    // Second, synchronize with publisher
    void *syncclient = zmq_socket (context, ZMQ_REQ);
    zmq_connect (syncclient, "tcp://localhost:5562");
}

```

```

// - send a synchronization request
s_send (syncclient, "");

// - wait for synchronization reply
char *string = s_recv (syncclient);
free (string);

// Third, get our updates and report how many we got
int update_nbr = 0;
while (1) {
    char *string = s_recv (subscriber);
    if (strcmp (string, "END") == 0) {
        free (string);
        break;
    }
    free (string);
    update_nbr++;
}
printf ("Received %d updates\n", update_nbr);

zmq_close (subscriber);
zmq_close (syncclient);
zmq_ctx_destroy (context);
return 0;
}

```

This Bash shell script will start 10 subscribers and then the publisher:

```

echo "Starting subscribers..."
for ((a=0; a<10; a++)); do
    syncsub &
done
echo "Starting publisher..."
syncpub

```

Which gives us this satisfying output:

```

Starting subscribers...
Starting publisher...
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates
Received 1000000 updates

```

We can't assume that the SUB connect will be finished by the time the REQ/REP dialog is complete. There are no guarantees that outbound connects will finish in any order

whatsoever, if you're using any transport except `inproc`. So, the example does a brute-force sleep of one second between subscribing and sending the REQ/REP synchronization.

A more robust model could be:

- Publisher opens PUB socket and starts sending “Hello” messages (not data).
- Subscribers connect to SUB socket, and when they receive a “Hello” message, they tell the publisher via a REQ/REP socket pair.
- When the publisher has had all the necessary confirmations, it starts to send real data.

Zero-Copy

ØMQ's message API lets you send and receive messages directly to and from application buffers without copying data. We call it *zero-copy*, and it can improve performance in some applications. Like all optimizations, use this when you know it helps, and measure before and after. Zero-copy makes your code more complex.

To do zero-copy, you use `zmq_msg_init_data()` to create a message that refers to a block of data already allocated on the heap with `malloc()`, and then you pass that to `zmq_msg_send()`. When you create the message, you also pass a function that ØMQ will call to free the block of data, when it has finished sending the message. This is the simplest example, assuming “buffer” is a block of 1,000 bytes allocated on the heap:

```
void my_free (void *data, void *hint) {  
    free (data);  
}
```

```
// Send message from buffer, which we allocate and ØMQ will free for us  
zmq_msg_t message;  
zmq_msg_init_data (&message, buffer, 1000, my_free, NULL);  
zmq_msg_send (socket, &message, 0);
```

message 直接用 buffer 内的 data 送出
就用 my_free 将 buffer free 掉

There is no way to do zero-copy on receive: ØMQ delivers you a buffer that you can store as long as you wish, but it will not write data directly into application buffers.

On writing, ØMQ's multipart messages work nicely together with zero-copy. In traditional messaging, you need to marshal different buffers together into one buffer that you can send. That means copying data. With ØMQ, you can send multiple buffers coming from different sources as individual message frames. Send each field as a length-delimited frame. To the application it looks like a series of send and receive calls, but internally the multiple parts get written to the network and read back with single system calls, so it's very efficient.

Pub-Sub Message Envelopes

In the pub-sub pattern, we can split the key into a separate message frame that we call an *envelope*. If you want to use pub-sub envelopes, make them yourself. It's optional, and in previous pub-sub examples, we didn't do this. Using a pub-sub envelope is a little more work for simple cases, but it's cleaner, especially for real cases, where the key and the data are naturally separate things.

→ 把 Publisher 的 message 封裝
讓 Subscriber 能選擇要接收 哪些 msg

Figure 2-15 shows is what a publish-subscribe message with an envelope looks like.

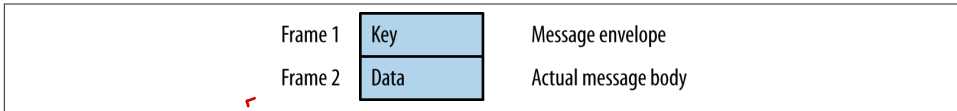


Figure 2-15. Pub-sub envelope with separate key

Recall that subscriptions do a prefix match. That is, they look for “all messages starting with XYZ.” The obvious question is: how to delimit keys from data so that the prefix match doesn't accidentally match data. The best answer is to use an envelope, because the match won't cross a frame boundary.

Here is a minimalist example of how pub-sub envelopes look in code. This publisher (Example 2-15) sends messages of two types, A and B. The envelope holds the message type.

Example 2-15. Pub-sub envelope publisher (*psenvpub.c*)

```
//  
// Pub-sub envelope publisher  
// Note that the zhelpers.h file also provides s_sendmore  
//  
#include "zhelpers.h"  
  
int main (void)  
{  
    // Prepare our context and publisher  
    void *context = zmq_ctx_new ();  
    void *publisher = zmq_socket (context, ZMQ_PUB);  
    zmq_bind (publisher, "tcp://*:5563");  
  
    while (1) {  
        // Write two messages, each with an envelope and content  
        s_sendmore (publisher, "A");  
        s_send (publisher, "We don't want to see this");  
        s_sendmore (publisher, "B");  
        s_send (publisher, "We would like to see this");  
        sleep (1);  
    }  
    // We never get here, but clean up anyhow
```

```

    zmq_close (publisher);
    zmq_ctx_destroy (context);
    return 0;
}

```

The subscriber, shown in [Example 2-16](#), wants only messages of type B.

Example 2-16. Pub-sub envelope subscriber (psenvsub.c)

```

//
// Pub-sub envelope subscriber
//
#include "zhelpers.h"

int main (void)
{
    // Prepare our context and subscriber
    void *context = zmq_ctx_new ();
    void *subscriber = zmq_socket (context, ZMQ_SUB);
    zmq_connect (subscriber, "tcp://localhost:5563");
    zmq_setsockopt (subscriber, ZMQ_SUBSCRIBE, "B", 1);

    while (1) {
        // Read envelope with address
        char *address = s_recv (subscriber);
        // Read message contents
        char *contents = s_recv (subscriber);
        printf ("%s] %s\n", address, contents);
        free (address);
        free (contents);
    }
    // We never get here, but clean up anyhow
    zmq_close (subscriber);
    zmq_ctx_destroy (context);
    return 0;
}

```

When you run the two programs, the subscriber should show you this:

```

[B] We would like to see this
[B] We would like to see this
[B] We would like to see this
[B] We would like to see this
...

```

This example shows that the subscription filter rejects or accepts the entire multipart message (key plus data). You won't get part of a multipart message, ever.

If you subscribe to multiple publishers and you want to know their addresses so that you can send them data via another socket (and this is a typical use case), create a three-part message, as illustrated in [Figure 2-16](#).

Frame 1	Key	Subscription key
Frame 2	Address	Address of publisher
Frame 3	Data	Actual message body

Figure 2-16. Pub-sub envelope with sender address

High-Water Marks

用於標示 socket 的 sending/receiving buffer (pipe) 是否滿了

When you can send messages rapidly from process to process, you soon discover that memory is a precious resource, and one that can be trivially filled up. A few seconds of delay somewhere in a process can turn into a backlog that blows up a server unless you understand the problem and take precautions. 積壓 崩潰

The problem is this: if you have process A sending messages to process B, which suddenly gets very busy (garbage collection, CPU overload, whatever), then what happens to the messages that process A wants to send? Some will sit in B's network buffers. Some will sit on the Ethernet wire itself. Some will sit in A's network buffers. And the rest will accumulate in A's memory. If you don't take some precaution, A can easily run out of memory and crash. It is a consistent, classic problem with message brokers.

What are the answers? One is to pass the problem upstream. A is getting the messages from somewhere else, so tell that process to stop, and so on up the line. This is called flow control. It sounds great, but what if you're sending out a Twitter feed? Do you tell the whole world to stop tweeting while B gets its act together?

Flow control works in some cases, but not in others. The transport layer can't tell the application layer to "stop" any more than a subway system can tell a large business, "Please keep your staff at work for another half an hour. I'm too busy."

The answer for messaging is to set limits on the size of buffers, and then, when we reach those limits, to take some sensible action. In some cases (not for a subway system, though), the answer is to throw away messages. In others, the best strategy is to wait.

ØMQ uses the concept of high-water mark (HWM) to define the capacity of its internal pipes. Each connection out of a socket or into a socket has its own pipe, and HWM, for sending and/or receiving, depending on the socket type. Some sockets (PUB, PUSH) only have send buffers. Some (SUB, PULL, REQ, REP) only have receive buffers. Some (DEALER, ROUTER, PAIR) have both send and receive buffers.

In ØMQ v2.x, the HWM was infinite by default. In ØMQ v3.x, it's set to 1,000 by default, which is more sensible. If you're still using ØMQ v2.x, you should always set an HWM on your sockets, be it 1,000 to match ØMQ v3.x, or another figure that takes into account your message sizes.

When your socket reaches its HWM, it will either block or drop data, depending on the socket type. PUB and ROUTER sockets will drop data if they reach their HWM, while other socket types will block.

Over the inproc transport, the sender and receiver share the same buffers, so the real HWM is the sum of the HWMs set by both sides.

Lastly, the high-water marks are counted in *message parts*, not whole messages. If you are sending two-part messages, the default HWM is 500. When you use the ROUTER socket type (discussed in detail in the next chapter), every message is at least two parts.

Missing Message Problem Solver

As you build applications with ØMQ, you will come across this problem more than once: losing messages that you expect to receive. We have put together a diagram (Figure 2-17) that walks through the most common causes for this.

If you're using ØMQ in a context where failures are expensive, you'll want to plan properly. First, build prototypes that let you learn and test the different aspects of your design. Stress them until they break, so that you know exactly how strong your designs are. Second, invest in testing. This means building test frameworks, ensuring you have access to realistic setups with sufficient computer power, and taking the time or getting help to actually test seriously. Ideally, one team writes the code and a second team tries to break it. Lastly, do get your organization to **contact iMatix** to discuss how we can help to make sure things work properly, and can be fixed rapidly if they break.

In short, if you have not proven that an architecture works in realistic conditions, it will most likely break at the worst possible moment.

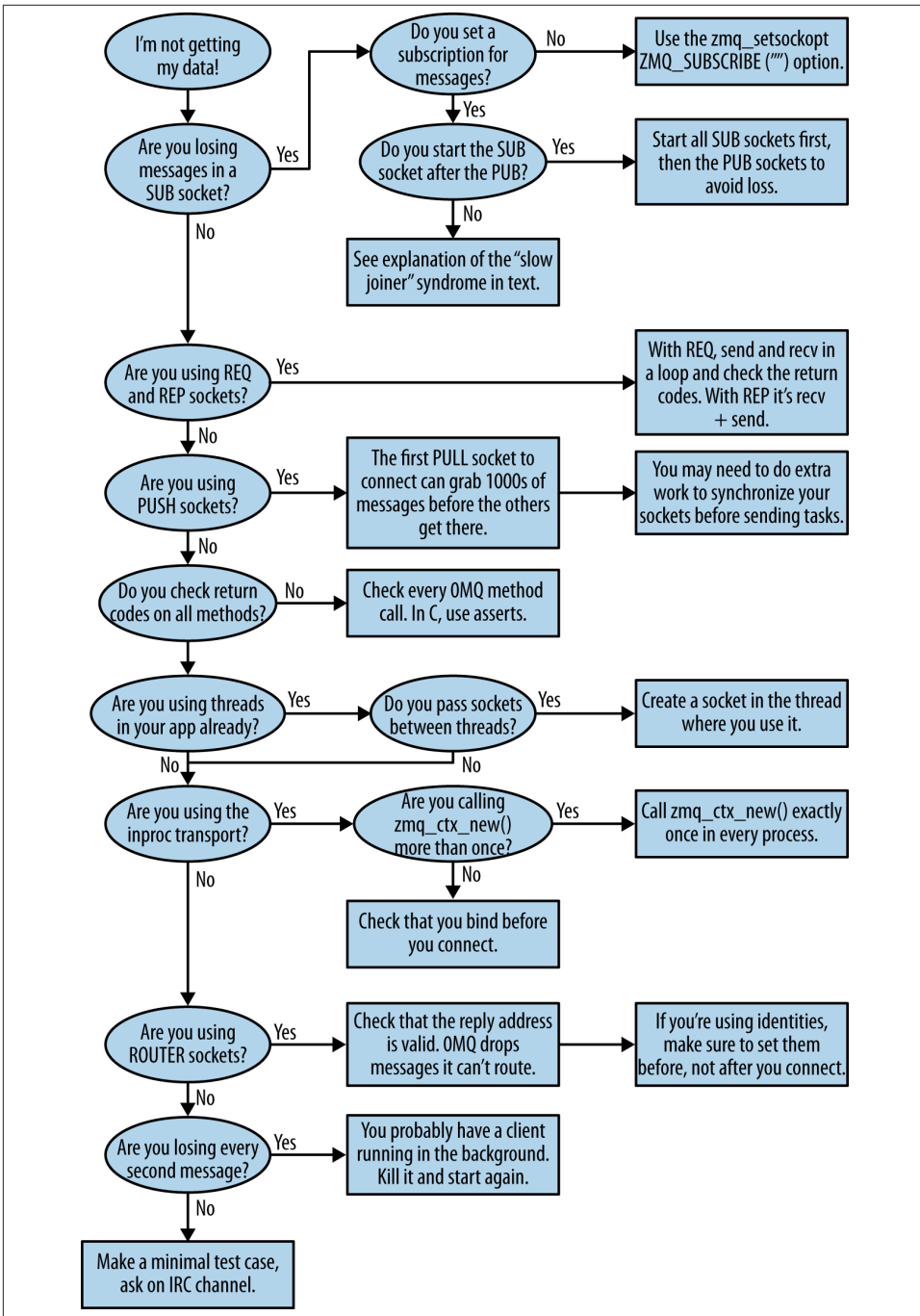


Figure 2-17. Missing message problem solver